

Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
Homework 3 (Due 9/29)

For the first two questions below (Verilog based), paste your entire Verilog code in a single text/doc file and submit the text/doc file on Canvas. Note that these Verilog questions need to follow the Verilog rules specified in the 552 project/Verilog rules document that can be found on Canvas.

1. Parallel Sub-Word Add

In this homework, you will design a 16-bit parallel sub-word addition instruction: PSA.

Its operation is described below:

The PSA instruction performs four half-byte signed additions in parallel to realize sub-word parallelism. Specifically, each of the four half bytes (4-bits) will be treated as separate numbers, but stored in whole as a single vector. When PSA is performed, the four numbers will be added separately. To be more specific, let the contents in rs and rt be aaaa_bbbb_cccc_dddd and eeee_ffff_gggg_hhhh, respectively, where a, b, c, d, e, f, g and h are 4-bit signed integers. Then after execution of PSA, the contents of rd will be {(aaaa+eeee), (bbbb+ffff), (cccc+gggg), (dddd+hhhh)}. You will need to check for overflow in each of the additions. If any of the sub-word additions result in an overflow, the PSA error flag should be set.

Submit the verilog code and a testbench. The module template is provided below.

```
module psa_16bit (  
    input  [15:0] a, // input A  
    input  [15:0] b, // input B  
    output [15:0] sum, // sum output  
    output          error // indicates 1+ overflows  
);  
...  
endmodule // psa_16bit
```

2. Shifter

Design a combinational 16-bit shifter to implement the following two shift instructions: SLL (Shift Left Logical) and SRA (Shift Right Arithmetic). The 4-bit shift amount is an unsigned binary number.

You are required to provide a Verilog design and testbench. The design interface to use is:

```
module shifter (  
    input [15:0] shift_in, // data to be shifted  
    input [3:0]  shift_val, // shift amount  
    input        mode,      // 0==SLL, 1==SRA  
    output [15:0] shift_out // data result of shift  
);  
...  
endmodule
```

The following question is non-Verilog. Submit this as a text doc or a snapshot.

Refer to COD Figure 4.24.

100011 00011 00010 0000000000010110 → R[rt] = M[R[rs]+SigExtImm
lw rs=3 rt=2 22

\$1	\$2	\$3	\$4	\$5	\$6	\$8	\$12	\$31	pc
-1	2	88	24	10	-6	8	2	-16	44

(a) What are the outputs of the sign-extend and the jump "Shift left 2" unit (near the top of COD Figure 4.24; the simple control and datapath are extended to handle the jump instruction) for this instruction word?

Sign-extend	Shift left 2 unit
000000000000000000000000010110	0001100010000000000001011000

Answer:

ALUOp[1:0]	Instruction[5:0]
00	010110

Answer:

New PC	Path
PC + 4 = 48	PC to ADD (PC+4) to branch MUX to jump MUX to PC

- (d) For each Mux, show the values of its data output during the execution of this instruction (given the current values of registers).

Answer:

RegDst Mux	ALUSrc Mux	MemToReg Mux	Branch Mux	Jump Mux
2	22	110	48	48

- (e) For the ALU and the two adder units, what are their data input values?

Answer:

ALU	Add (PC + 4)	Add (Branch)
88 and 22	44 and 4	48 and 22*4

- (f) What are the values of all inputs to the Register File?

Answer:

Read Register 1	Read Register 2	Write Register	Write Data	RegWrite
3	2	2	110	1